

Brain MRI Tumor Segmentation and Quantitative Analysis Using Classical Image Processing

Week 5 – Conversion of Image to Binary using Python

Kush Kelaiya

Enrollment No.: AU2440018

*School of Engineering and Applied Science
Ahmedabad University
Ahmedabad, Gujarat, India*

Rushin Modi

Enrollment No.: AU2440028

*School of Engineering and Applied Science
Ahmedabad University
Ahmedabad, Gujarat, India*

Kritee Shah

Enrollment No.: AU2440080

*School of Engineering and Applied Science
Ahmedabad University
Ahmedabad, Gujarat, India*

Jash Odedara

Enrollment No.: AU2440096

*School of Engineering and Applied Science
Ahmedabad University
Ahmedabad, Gujarat, India*

Submitted By:- G9 - The Medics.

Repo Link:-

https://github.com/Rushin-Mod/DIP_2026_P20_G9

I. WORK DONE THIS WEEK:-

This week we managed to convert the entire image series into binary. We have used specific libraries in order to achieve this. We have yet again used the same principle (trying to simplify the filtering that we have applied) we have been using throughout the course, but we also tried to go for a simpler option, by using the filters that we have already done in the course. However, they do not actually work on the images that we are supposed to use, as the time series block of the entire reading cannot be directly managed.

So we have tried to look into it in a different manner. We have instead, via intuition and inference, recognised scans without a tumour at any spot and are treating them as a form of clean images. Via this, we are basically comparing them with other scans, and it has a high likelihood of noticing major clots and clusters (tumours) formed within the brain. However, this does not have a complete guarantee of filtering everything out perfectly and might cause an error according to the selection.

Coming back to what we primarily did this week, we tried to convert the entire scan read into binary so that it can be processed more efficiently. The image shape is 144 slices and 256×256 , so this whole data, whenever we tried to run it on Python, caused most PCs/laptops to crash when we tried to execute it directly on Python. We also had to convert the data type, which might cause some discrepancies, but that cannot be avoided to actually deal with the computation. The conversion was from `float64` to unsigned 16-bit integer (`uint16`), but it is not causing any data loss from what we could notice.

This conversion was done for the work that we want to do next week.

We have also provided the facility to go slice by slice for the entire scan to better prevent false positives, just in case, and also to avoid false negatives. We have converted the three different views accordingly and compared all views for the same reading to make sure of our reading. Going through all the slices gives better results: a clot is clearly visible in many of the images, so we have to set up a method to detect this spike in thickness at a few places, creating white/bright spots.

We accordingly checked the voxel matrix, and through that we realised that due to the multi-matrix size, the total number of voxels would go up to 9437184 for each individual scan, resulting in our initial conundrum with processing. Therefore, we have converted the data into binary form because we further plan to do the rest of the computation using C++, as it is showing more chances of running overall than whatever we have tried with Python yet.

II. WORK TO BE DONE NEXT WEEK:-

As previously mentioned, we have gone through an entire MRI scan and tried to process it with Python but faced many difficulties. Accordingly, we now plan to convert the data into pure binary, via which it would be easier to process in C++. We necessarily have to do that because we do not know how to perform image processing with Python, so we are using the strengths of both languages so that we can utilise and optimise the work of our project.

We further plan to first try to implement this over one scan, and then extend it to run on more scans, comparing the actual accuracy of these scans with what we are aiming to do. We will compare this with what we have initially done in Python and check whether some implementation can be interchanged

between the two languages. Currently, we are looking into this and trying to understand the best path forward.

Further, we are going to improve the segmentation process and try to implement it for multiple scans, going through this and understanding how to make any changes wherever required and planning accordingly. We are also looking into any further methods to convert code from Python or C++, if any conversion is possible for the various functions. We plan to further try to apply the concepts that we have learnt in the course to better implement this.